

Game Programming with SDL

Ranim Tom

April 14, 2026

Contents

1	Game Development with C++ and Lua	7
1.1	Introduction	7
1.2	Installation and Configuration	9
1.2.1	C/C++	9
1.2.2	SDL Installation	9
1.2.3	Lua	9
1.2.4	Command Exploring directory structure	9
1.3	Building	9
1.3.1	Makefile	9
1.4	Compiling and testing all dependencies	10
1.4.1	sol and lua libraries	10
1.4.2	Binary Libraries	10
1.4.3	Static and Dynamic Libraries	10
1.5	Summary Game Development with C++	12

Todo list

Outcomes and deliverable	8
------------------------------------	---

Chapter 1

Game Development with C++ and Lua

1.1 Introduction

In this chapter, we build a simple 2D game but nevertheless functional game with C++. We will go in this section about the outcomes and the deliverable of this project.

1. Game Engine Architecture:

We discuss programming pattern used inside a game engine. So it's not about making games only (like drawing some png structures and making them move), no it's about an application called game engine at 1st, then using this engine to make many game.

2. Creating a final game:

To fix the concepts and the idea and to understand more, we will also create a game using (where we have all the idea of attack, collision, motion, ...) this engine. Also, we will use Lua (a programming scripting language) to enable the users and the level designers to change some of the behavior of the game. An example is shown [Figure 1.1](#), where the Lua file is used to set some parameters of the game



Figure 1.1: Goal of this project in this chapter

3. Use modern C++ and other helper libraries:

We will learn throughout the course the use of modern C++ (like templates, smart pointers, ...).

Also we will use some other *helper libraries*: SDL for graphics, audio, ..., ImGui to create application, glm for math manipulation (such as matrix operation, ...), and finally Lua.

4. Outcomes and deliverable: to repeat later the intro from the implementation point, and write down the role of each concept.

comes and
verable

1.2 Installation and Configuration

Now in this section we will see how to install and configure the necessary libraries, dependencies, and make sure that everything run well.

1.2.1 C/C++

See ??.

1.2.2 SDL Installation

- `sudo apt install libsdl2-dev`
 - This is the core of the SDL library.

But this is not enough, we need also some *extensions*

- `sudo apt install libsdl2-image-dev` (to load png images,...)
- `sudo apt install libsdl2-ttf-dev` (work with fonts)
- `sudo apt install libsdl2-mixer-dev` (for sounds)

1.2.3 Lua

```
sudo apt install liblua5.3-dev
```

1.2.4 Command Exploring directory structure

A nice command to explore the structure of some project, we can use `tree -d`, and it will give the hierarchy of our folder in a tree structure, with all the directory in it.

1.3 Building

Now once we install things, we need to test if our installation is correct. We will use `makfile` to automate our compilation, linking and running process.

1.3.1 Makefile

Makefile are files processed (read) by a tool named `make`, which is given from **gnu** community (and directly installed when we ran `build-essential`).

When defining rule with **Makefile**, it is necessary to indent the rule with a tab.

1.4 Compiling and testing all dependencies

Till now, we compiled and verified that C++ and SDL (along with extensions) are compiled correctly.

However, we need also to test and see the compilation for the 3rd party libraries installed inside the `lib` folder in our project.

Unlike SDL, which is installed *globally in our linux system*, the libraries in `lib` folder are installed *locally inside our project*, so we need to instruct the compiler to search inside the `lib` folder, to find the necessary header files.

1.4.1 sol and lua libraries

For `sol` library, we added also extra linker flags in order to have the compiler to link between `ol` `lua`

1.4.2 Binary Libraries

When we asked the package manager (`apt`) to install SDL and `lua`, it will install *binary version* of these libraries suitable for our OS (in my case `Linux`). So every time we run `make build`, we don't compile all the source code of SDL and `lua`, we only compile our source code, then when reaching the linking stage, the linker fetch the binary of these libraries.

This will save a huge compile time, because compilation time can be headache in C/Cpp.

1.4.3 Static and Dynamic Libraries

One final topic that might be useful to discuss is the difference between `.lib` and `.dll` files.

This small article is mostly for curiosity purposes so we understand what programmers are talking about when they mention statically linked and dynamically linked libraries.

Libraries are great because they can have code that we want to use in multiple programs. For example, we can create a function that knows how to count how many words are in a string, and that function might be useful in lots of different programs. Once we get the function to work correctly, we don't need to recompile it every time, so we we put the binary of that code in a library and the linker can extract and insert the compiled code into our current program during the linking stage. This is what we call static linking.

Libraries can either be Static (`LIB`) or Dynamic (`DLL`). Let us look at both options and list their main differences.

Statically Linked Libraries: `LIB` files are compiled but not linked. We link them statically during your program's final link step. They are static because they cannot be shared among programs. Once you link them with a program, they will be used by that program only. These files usually have the extension `.lib` or `.a`. (where the `a` stands for *archive* and it's the Unix equivalent of a Windows `.lib`).

Dynamically Linked Libraries: DLL files take the idea of libraries one step further. It seems wasteful to have multiple copies of the library, each one taking up space in each of the programs. DLLs are dynamically linked, meaning that we can share one copy of the function with several programs. Multiple programs running at the same time (that use the same function) can share one copy of that function, saving memory. These dynamically linked libraries usually have the extension `.dll` on Windows or `.so` on UNIX (*shared object*).

1.5 Summary Game Development with C++